



SURF

WFO Updates

WFO Code Sprints & Architecture, Documentation

Mark Moes @ SURF

September 22, 2026

Dublin

WFO Code Sprints

WFO Partner Code Sprint

Add status update

Insights

Workflows

Issues

Previous iteration

Current iteration

Next iteration

Team capacity

Roadmap

My items

Berkeley

Architecture

+ New view

-status:Done

84

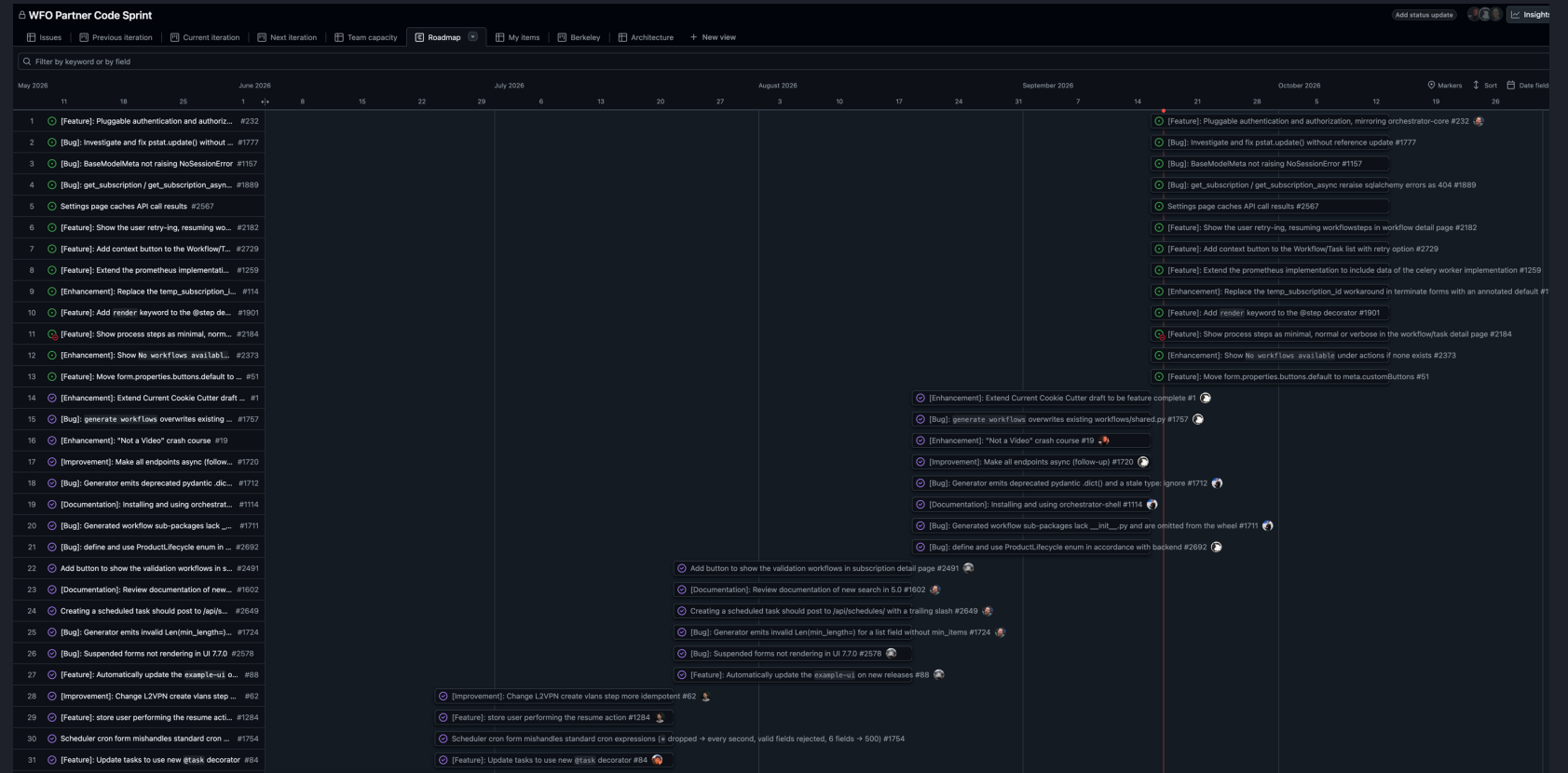
Title	Parent issue	Value	Effort	Created	Assignees	Status	Iteration	Labels
[Bug]: get_subscription / get_subscription_async raise sqlalchemy er... #1889	EPIC: backend #1706	Medium	Low	Sep 3, 2026			Iteration 21	bug priority: low
Settings page caches API call results #2567	EPIC: frontend #1707	Medium	Medium	May 27, 2026			Iteration 21	bug enhancement frontend
[Feature]: Add context button to the Workflow/Task list with retry option #2729	EPIC: frontend #1707	Medium	Low	Sep 10, 2026			Iteration 21	enhancement good first issue
[Feature]: Pluggable authentication and authorization, mirroring orchestr... #232	EPIC: backend #1706	High	Medium	Aug 20, 2026			Iteration 21	enhancement
[Feature]: Extend the prometheus implementation to include data of the... #1259	EPIC: backend #1706	Medium	Low	Dec 1, 2025			Iteration 21	feature
[Enhancement]: Replace the temp_subscription_id workaround in termin... #114	EPIC: example orchestrator ...	Medium	Low	Jun 25, 2026			Iteration 21	enhancement good first issue
[Feature]: Add render keyword to the @step decorator #1901	EPIC: backend #1706	Medium	Medium	Sep 10, 2026			Iteration 21	feature
[Feature]: Show process steps as minimal, normal or verbose in the wor... #2184	EPIC: frontend #1707	Medium	High	Sep 24, 2025			Iteration 21	code sprint 2026 WFO code sprint
[Enhancement]: Show No workflows available under actions if none ... #2373	EPIC: frontend #1707	Medium	Low	Dec 9, 2025			Iteration 21	enhancement good first issue
[Feature]: Move form.properties.buttons.default to meta.customButtons #51	EPIC: maintenance #1708	Medium	Low	Nov 20, 2025			Iteration 21	enhancement
[Feature]: Add PolyNSI and SuPA to the example orchestrator #44	EPIC: example orchestrator ...	Low	Medium	Sep 11, 2025	hanstro...		Iteration 22	
----- epics -----				Mar 3, 2025				
EPIC: documentation #871		High	High	Mar 3, 2025	slbyrne...	In Progress		code sprint 2026 documentation triage
EPIC: example orchestrator #1574				Apr 16, 2026				
EPIC: backend #1706				Jun 17, 2026				
EPIC: frontend #1707				Jun 17, 2026				
EPIC: maintenance #1708				Jun 17, 2026				
EPIC: NSI features #36				Jul 6, 2026				
EPIC: orchestrator-core parallel steps #870		High	High	Mar 3, 2025	hanstro...			code sprint 2026
----- issues -----				Mar 3, 2025				
[Feature]: Add low priority documentation for Pydantic Forms Library #538	EPIC: documentation #871	High	Medium	Mar 7, 2024				documentation feature
[Feature]: Add low priority documentation for Redis and Deployment Met... #537	[Documentation]: Improve L...	High	Medium	Mar 7, 2024				documentation feature
[Feature]: Document possible values from ProcessStatus #969	EPIC: documentation #871	High	Medium	Jun 11, 2025				documentation
[Feature]: Add misc. low priority documentation #534	EPIC: documentation #871	High	Medium	Mar 7, 2024				documentation feature priority: low triage
[Improvement]: Remove deprecated code/documentation for scheduler.... #1346	EPIC: maintenance #1708	Medium	Low	Jan 19, 2026				documentation maintenance triage
[Maintenance]: Remove property "uniforms" #55	EPIC: maintenance #1708	Medium	Low	Feb 16, 2026				breaking change maintenance
[Maintenance]: Convert to a namespaced package #35	EPIC: maintenance #1708	Medium	Low	Jun 3, 2026				maintenance
[Enhancement]: Make the generated license header configurable and a... #1714	EPIC: backend #1706	Medium	Low	Jun 18, 2026				enhancement good first issue
[Maintenance]: Remove forms property "uniforms" #1416	EPIC: maintenance #1708	Medium	Low	Feb 19, 2026				breaking change maintenance triage
[Improvement]: Add PAUSED state and implement state machine #1014	EPIC: backend #1706	Medium	Medium	Jul 24, 2025				enhancement

- Monthly backlog labeling and prioritization (SURF, ESNET, GEÁNT)
- Weekly story refinement (SURF)

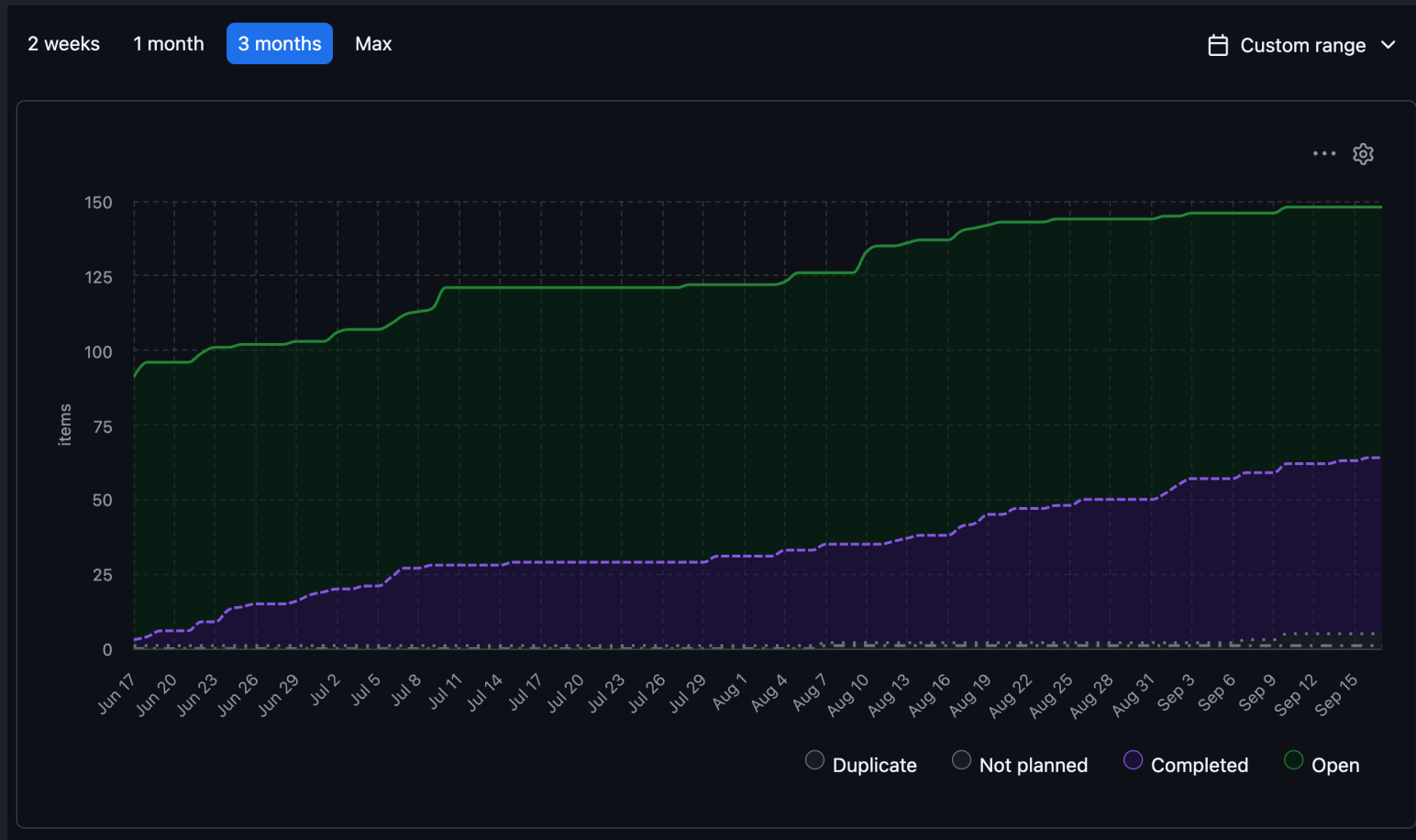


WFO Code Sprints

- Bi-weekly WFO Sprint planning and ceremonies (SURF)



WFO Code Sprints



- Steady progress over the last 3 months



WFO Architecture Sync



- Participants: certified architects from SURF, ESNET, GEÁNT

WFO Architecture Sync

- Monthly high-level discussion on impactful / fundamental topics
 - MCP server
 - Cookiecutter
 - Parallelism
 - Event streaming / notifications

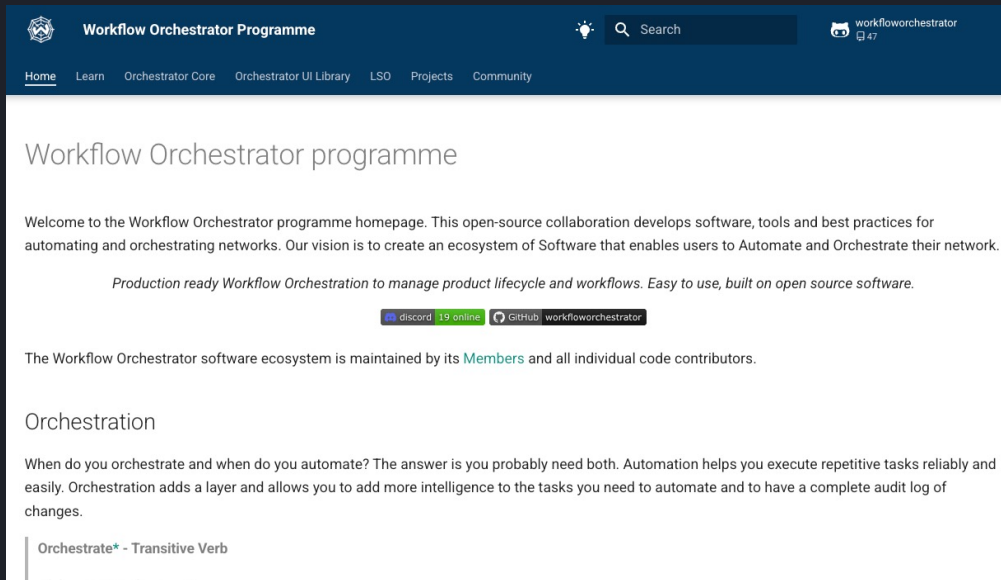
[Sep 1, 2026 WFO Architecture Sync	
Attendees: Peter Boers karel.vanklink@geant.org mark.moes@surf.nl Alexander Ray	
Meeting Agenda	
1. Review Alex's work on designing streaming notifications (Actual POC)	
a. Use redis + celery instead of wrapping ORM, creating new event registry	
b. Make sure to consider scheduler tasks	
c. Look at Topic (Mark)	
2. Discuss Abstract Product Blocks (Karel) Don't nest, instead treat as "mixins" when part of plugin	
a. Need generic "Node" for optical module to avoid ProductNode/OpticalNode/etc.	
b. Dependency chain/Concrete block inheritance not possible due to assumptions in core relationship serialization	
c. Ended up shipping models as part of the optical plugin (i.e., OpticalBlock gets added to downstream user model)	
3. Distributing unit test helpers from orchestrator-core (Ticket: Made 🟢)	
5-Updating the documentation structure	
a. Maybe also switch to an alternative as Markdown is going out of support as-is	
Notes	
•	
Action Items	
□ Dev days demo agentic orchestrator!	
Future Discussions	
• Agentic orchestrator will be demoed w/ example orchestrator at dev days	
• Updating the documentation structure	
o Maybe also switch to an alternative as Markdown is going out of support as-is	
Aug 18, 2026 WFO Architecture Sync	
Attendees: Peter Boers karel.vanklink@geant.org mark.moes@surf.nl Alexander Ray	
Meeting Agenda	
6. Review Karel's "Cookie cutter" python templates for creating plugin template	
7. MCP (as plugin?)	
a. Install optional [POC] dependencies & enable boolean	
b. It's maintained as part of orchestrator-core	
i. Decide where the line is drawn 🟡 "use Ennio Morricone Good bad and ugly	
c. Part of REST API	
8. Check app/dependencies labelled issues	
9-Review Alex's work on designing streaming notifications? (Actual POC)	
a. ...next time	
Notes	
• Created Cookie Cutter repo issues with some to-do items	
• MCP already has a separate docker image	
• Developer Day! GRNet coming?	
Action Items	
□ Write the docs for MCP server peter.boers@surf.nl	
□ Follow up with CESnet, GRNet, NCSC for Dev Days	
Future Discussions	
• Distributing unit test helpers from orchestrator-core	
• Updating the documentation structure	
o Maybe also switch to an alternative as Markdown is going out of support as-is	
• Some orchestrator-core edits probably needed to support orchestrator-optical	
o Abstract product blocks only work on a single level	
o Abstract product blocks as a resource type not supported	
o Nested abstract product blocks not supported	
Jul 7, 2026 WFO Architecture Sync	
Attendees: Alexander Ray (organizer), mark.moes@surf.nl, karel.vanklink@geant.org, and Peter Boers	
Meeting Agenda	
10. Follow-up on Parallel Workflows decision	
a. Presentation from Peter on deciding the fate of parallel workflows	
b. Review documentation re: EShet parallel workflows (Alex)	
c. Review documentation re: GEANT parallel workflows (Karel)	
11. Review Alex's work on designing streaming notifications	
Notes	

Q label:architecture -status:Done	
Title	
1	[Enhancement]: Passing data through form fields changes their requiredness #9
2	[Feature]: Using model-validators to perform validations across products #283
3	[Feature]: Efficient way to retrieve other subscription data in computed_fields #899
4	[Enhancement]: Add orchestrator-worker to example orchestrator #46
5	[Feature]: Keep history of specific tasks #1279
6	[Feature]: add a generic notification mechanism #1511
7	[Feature]: Ability to lock steps/resources to avoid concurrency issues #1647
8	[Feature]: Think of a better name for product.type #1756
9	[Feature]: Support nested abstract product types and blocks #1769



WFO Website updates

- Restructured home page



The screenshot shows the homepage of the Workflow Orchestrator Programme. The header is dark blue with the logo, title, search bar, and navigation links. The main content area is white and includes a welcome message, a quote about production-ready orchestration, social media links, and a section on orchestration.

Workflow Orchestrator Programme

Home Learn Orchestrator Core Orchestrator UI Library LSO Projects Community

Workflow Orchestrator programme

Welcome to the Workflow Orchestrator programme homepage. This open-source collaboration develops software, tools and best practices for automating and orchestrating networks. Our vision is to create an ecosystem of Software that enables users to Automate and Orchestrate their network.

Production ready Workflow Orchestration to manage product lifecycle and workflows. Easy to use, built on open source software.

[discord](#) [19 online](#) [GitHub](#) workfloworchestrator

The Workflow Orchestrator software ecosystem is maintained by its [Members](#) and all individual code contributors.

Orchestration

When do you orchestrate and when do you automate? The answer is you probably need both. Automation helps you execute repetitive tasks reliably and easily. Orchestration adds a layer and allows you to add more intelligence to the tasks you need to automate and to have a complete audit log of changes.

Orchestrate* - Transitive Verb

Project Goal

The **Workflow Orchestrator** provides a framework through which you can manage service orchestration for your end-users. The framework helps and guides you, the person who needs to get things done, through the steps from automation to orchestration. With an easy to use set of API's and examples, you should be up and running and seeing results, before you completely understand all ins and outs of the project. The Workflow Orchestrator enables you to define products to which users can subscribe, and helps you intelligently manage the lifecycle, with the use of **Creation, Modification, Termination and Validation** workflows, of resources that you provide to your users.

What does a workflow look like? It must be pretty complex!

Programming a new workflow should be really easy, and at its core it is. By defining workflows as Python functions, all you need to do is understand how to write basic python code, the framework will help take care of the rest.

```
@workflow("Name of the workflow", initial_input_form=input_form_generator)
def workflow():
    return (
        init
        >> arbitrary_step_func_1
        >> arbitrary_step_func_2
        >> arbitrary_step_func_3
        >> done
    )
```

If this has sparked your interest, read more about the [framework](#) or jump into [Getting Started](#).

Tooling

The WFO programme maintains an entire ecosystem of tooling, a non-comprehensive list in no particular order.

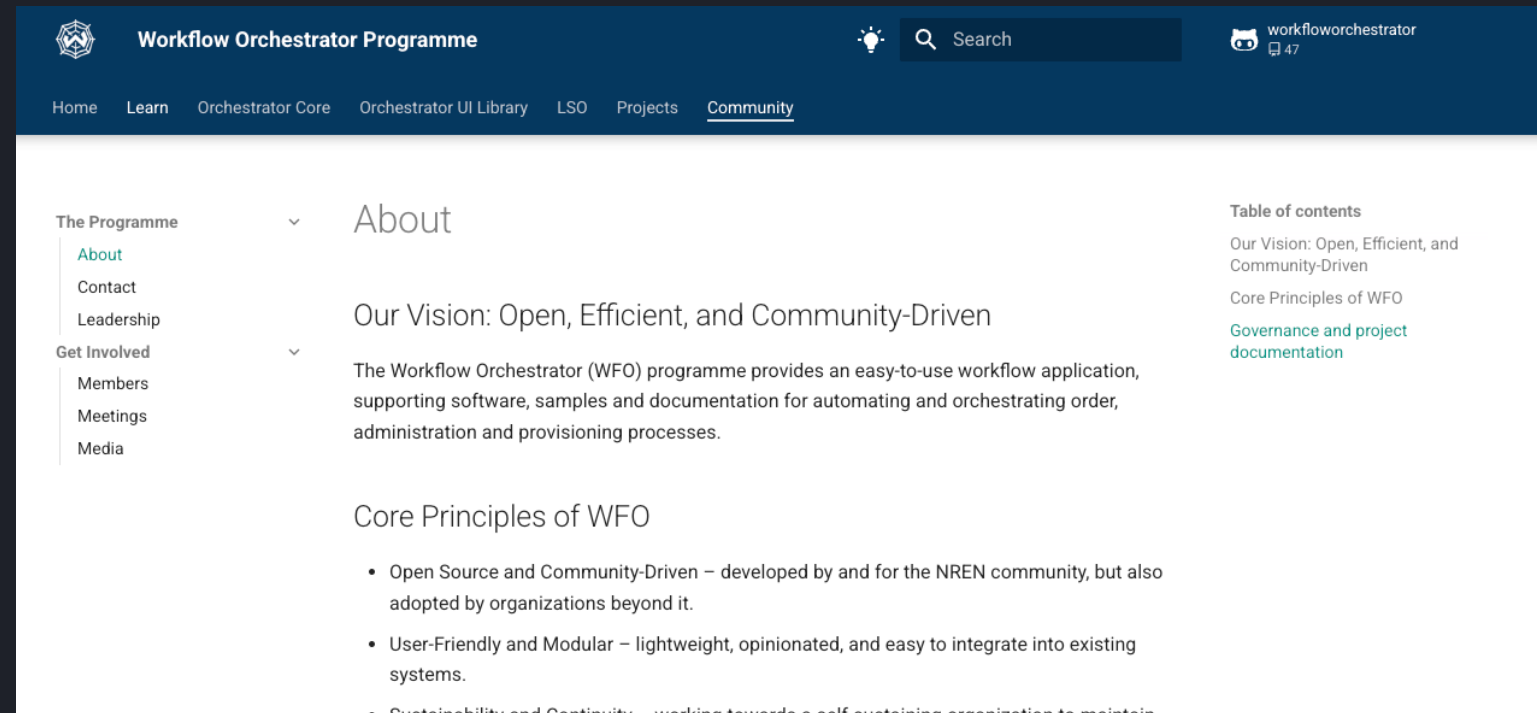
Orchestrator-Core Downloads Docs Issues This Python program leverages the power of FastAPI, Pydantic and SQLAlchemy to create an orchestration engine. Source · Docs	Orchestrator-UI Downloads Docs Issues Component library for a modern and flexible NextJS app on top of the Orchestrator-core. Source · Docs	Orchestrator Example UI Issues Example UI with a NextJS implementation of our component library. Source
LSO Downloads Docs Issues This application provides an API layer on top of Ansible playbooks. Source · Docs	Example Orchestrator Issues This repository houses a Docker-compose running a full stack of the Orchestrator, UI and Netbox. Source	Pydantic-Forms Downloads Docs Issues A library that includes standardized Python Form classes that can be used when generating form components from JSON-schema. Source · Docs
SuPA Issues An NSI Ultimate provider agent with a gRPC API. Source · Docs	PolyNSI Issues A bidirectional SOAP to gRPC translating proxy server for the NSI protocol. Source	

Which organizations are using Workflow Orchestrator?



WFO Website updates

- Horizontal tab layout to separate by content



WFO Website updates

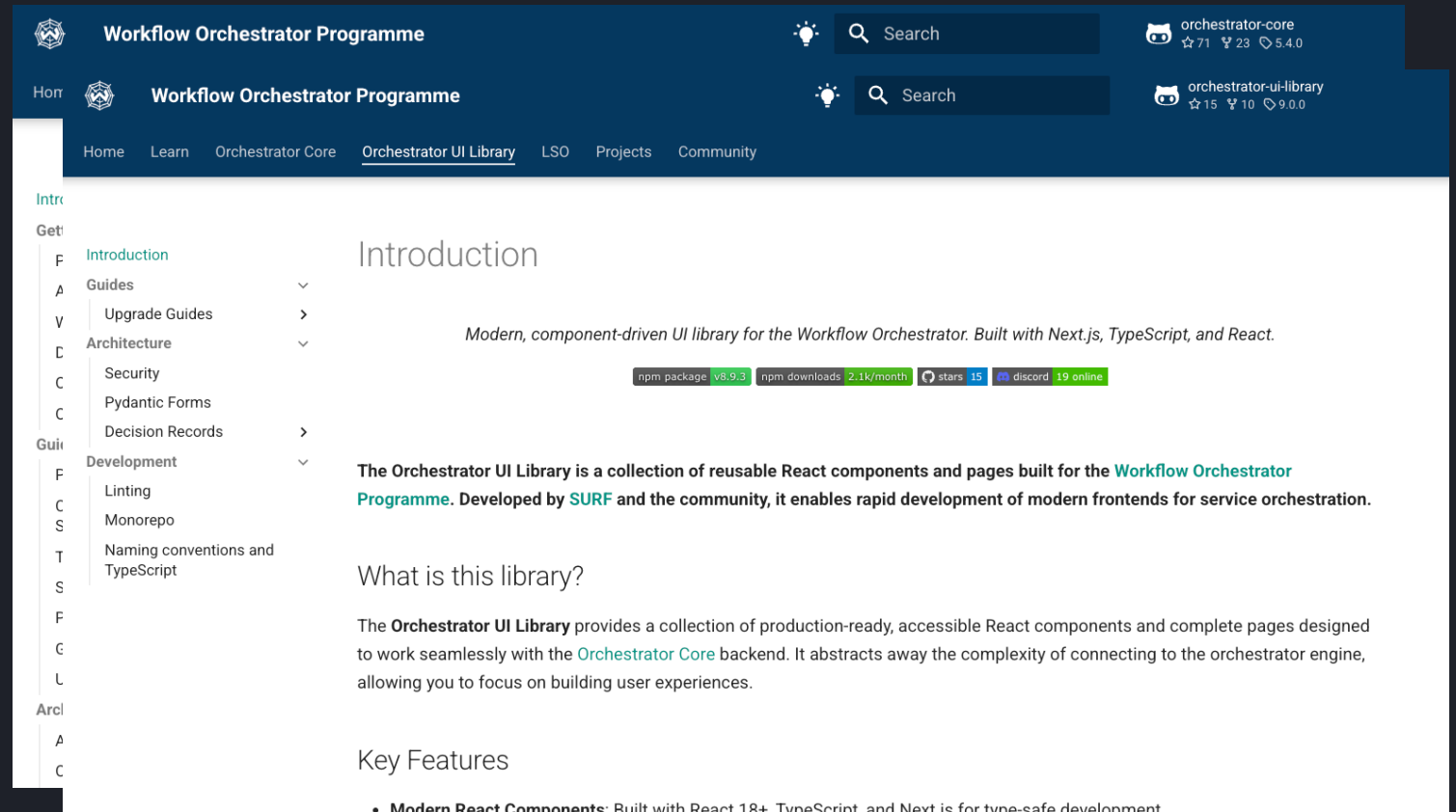
- New Learn page, previously “Architecture”
- Will be reworked further
- New: At a Glance!

The screenshot displays the 'Workflow Orchestrator Programme' website. The header includes the program name, a search bar, and a GitHub repository link. The main navigation bar lists 'Home', 'Learn', 'Orchestrator Core', 'Orchestrator UI Library', 'LSO', 'Projects', and 'Community'. The 'Learn' page is active, showing a sidebar with 'Concepts' (At A Glance, The Framework, Orchestrator UI, Input Forms) and 'Example Orchestrator Workshop' (Overview, Bootstrapping, Products, Workflows, Create your own product and workflows). The main content area is titled 'Workflow Orchestrator At A Glance' and features an 'Info' box stating the article is for potential adopters. Below this, the text describes the framework's purpose and its use in modeling subscription lifecycles. A 'Scenario: Adding a Link' section follows, with another 'Info' box explaining the end-user experience. The page concludes with a paragraph about establishing a core link between nodes in Amsterdam and London, and a note about creating a link via the 'New Subscription' dropdown. A footer bar shows social media links and a 'New Subscription' button.



WFO Website updates

- Updated Introduction pages



The screenshot displays the Workflow Orchestrator Programme website. The top navigation bar includes the logo, the title 'Workflow Orchestrator Programme', a search bar, and links to GitHub repositories 'orchestrator-core' and 'orchestrator-ui-library'. Below this, a secondary navigation bar lists 'Home', 'Learn', 'Orchestrator Core', 'Orchestrator UI Library' (which is underlined), 'LSO', 'Projects', and 'Community'. The main content area features a left sidebar with a tree view of the site's structure, including 'Introduction', 'Guides', 'Architecture', 'Security', 'Pydantic Forms', 'Decision Records', 'Development', 'Linting', 'Monorepo', 'Naming conventions and TypeScript', and 'Arc42'. The main content area is titled 'Introduction' and contains the following text: 'Modern, component-driven UI library for the Workflow Orchestrator. Built with Next.js, TypeScript, and React.' Below this text are statistics for the npm package: 'v8.9.3', '2.1k/month' downloads, '15' stars, and '19 online' on discord. The text continues: 'The Orchestrator UI Library is a collection of reusable React components and pages built for the Workflow Orchestrator Programme. Developed by SURF and the community, it enables rapid development of modern frontends for service orchestration.' This is followed by a section titled 'What is this library?' which states: 'The Orchestrator UI Library provides a collection of production-ready, accessible React components and complete pages designed to work seamlessly with the Orchestrator Core backend. It abstracts away the complexity of connecting to the orchestrator engine, allowing you to focus on building user experiences.' The final section is 'Key Features', which lists 'Modern React Components: Built with React 18+, TypeScript, and Next.js for type-safe development'.



WFO Website updates

- Added documentation of pydantic-forms

The screenshot displays the 'Workflow Orchestrator Programme' website. The top navigation bar includes links for Home, Learn, Orchestrator Core, Orchestrator UI Library, LSO, Projects, and Community. A search bar and a GitHub repository link for 'pydantic-forms' (14 stars, 0 forks, 2.6.0 version) are also present. The main content area is titled 'Reference' and provides an overview of public modules. It lists several forms: `DisplayOnlyFieldType`, `FormPage`, `list_forms`, `register_form`, `generate_form`, `post_form`, and `start_form`. It also lists async variants: `generate_form`, `post_form`, and `start_form`. A table of contents on the right lists various field types: `Callout`, `ContactPersonName`, `DisplaySubscription`, `Divider`, `Hidden`, `Label`, `ListOfOne`, `ListOfTwo`, `LongText`, `Markdown`, `MigrationSummary`, `OrganisationId`, `Timestamp`, `Accept`, `AcceptValues`, and `Choice`. The main content area also includes a section for `FormPage` with its base class `pydantic.BaseModel` and attributes `model_config` and `meta__`. The `list_forms` function is also documented.

Workflow Orchestrator Programme

Home Learn Orchestrator Core Orchestrator UI Library LSO Projects Community

Search

pydantic-forms
★14 🍴0 🔄2.6.0

Overview

Pydantic Forms

Introduction

Installation

Usage

Examples

Errors

How it works

Reference

Architecture Decisions

Reference

Every public module, listed in full. See [Usage](#) and [How it works](#) for the explanations behind them.

Forms

`class` `DisplayOnlyFieldType`

`class` `FormPage`

Bases: `pydantic.BaseModel`

`attr` `model_config` class-attribute instance-attribute

```
model_config = pydantic.ConfigDict(arbitrary_types_allowed=True, title='unknown')
```

`attr` `meta__` class-attribute

```
meta__: pydantic_forms.types.JSON = None
```

Data about the page itself, passed to the frontend alongside its JSON schema.

Set it on a subclass to tell the frontend something the schema cannot express, such as whether another page follows. It travels out as the `meta` key of the `FormNotCompleteError` response. Being a `ClassVar` it is not a form field, so it stays out of the schema and the validated result.

`func` `list_forms`

```
list_forms() -> list[str]
```

Table of contents

Forms

- `class` `DisplayOnlyFieldType`
- `class` `FormPage`
- `func` `list_forms`
- `func` `register_form`
- `func` `generate_form`
- `func` `post_form`
- `func` `start_form`

Async variants

- `func` `generate_form`
- `func` `post_form`
- `func` `start_form`

Field types

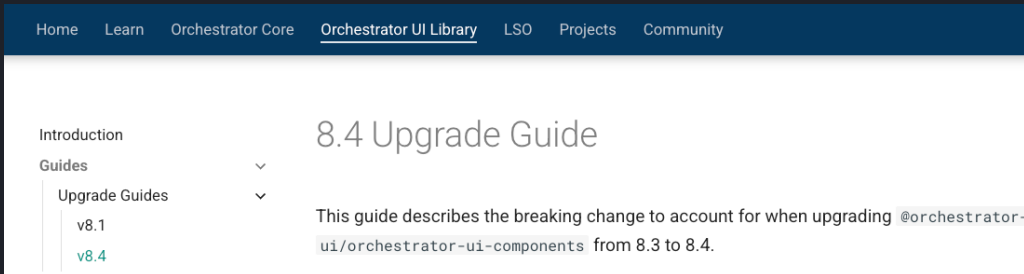
- `attr` `Callout`
- `attr` `ContactPersonName`
- `attr` `DisplaySubscription`
- `attr` `Divider`
- `attr` `Hidden`
- `attr` `Label`
- `attr` `ListOfOne`
- `attr` `ListOfTwo`
- `attr` `LongText`
- `attr` `Markdown`
- `attr` `MigrationSummary`
- `attr` `OrganisationId`
- `attr` `Timestamp`
- `class` `Accept`
- `class` `AcceptValues`
- `class` `Choice`

WFO Website updates

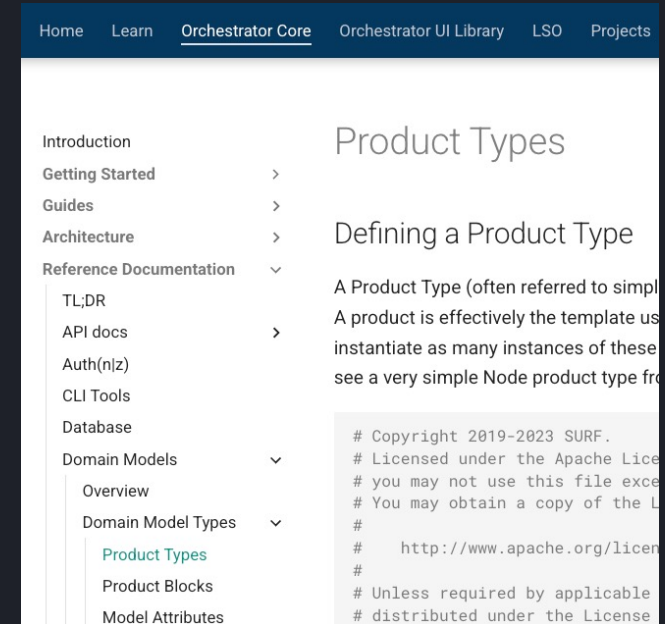
- Github repo facts per subpage

 orchestrator-core ☆ 71 🍴 23 📦 5.4.0	 orchestrator-ui-library ☆ 15 🍴 10 📦 9.0.0
 lso ☆ 15 🍴 2 📦 v5.1.0	 pydantic-forms ☆ 14 🍴 0 📦 2.6.0

- UI Upgrade Guides



- TOC folding and styling



WFO Website plans

- Improve the orchestrator-core Guides/Reference docs
 - Better separate “How to X” and “All info about Y”
 - Restructure guides to help users find what they need
- Rework the **Learn** section
 - Improve flow between pages
 - Provide high-level overview of components
 - Move implementation details to projects
- Replace mkdocs by something that’s maintained



The end.



SURF